

# Week 10: Lambda calculus and evaluation

Functional Programming in Lean

# Learning goals

- ▶ Read lambda terms as an inductive datatype
- ▶ Explain free versus bound variables
- ▶ Compare evaluation strategies on one term

# Central idea

A lambda term is a variable, abstraction or application.  
Binding makes substitution delicate.

# Lean example

```
inductive Term where  
  | var (index : Nat)  
  | lam (body : Term)  
  | app (function argument : Term)
```

# Evaluation with an environment

```
def evalCBV :
```

```
  Nat → List Value → Term → Option Value
```

A closure stores its body and defining environment. The fuel bounds evaluation of applications.

# What to notice

- ▶ De Bruijn index zero refers to the nearest binder
- ▶ A term can be checked for well-scoped variables
- ▶ Call-by-value and call-by-name can reduce different subterms first

# Live coding

1. Build the identity term
2. Complete `wellScoped`
3. Trace two reductions on paper before coding more

# Pause and predict

Is `Term.var 0` closed at depth zero?

# Exercise connection

Open `exercises/Week10.md` after the lecture.

The complete example is `Course/Week10.lean`; the fill-in version is `Templates/Week10.lean`.